



LECTURE 1

C++ Review + ADT Design

CS24 · Fall 2026 · Diba

CS24 Course Map

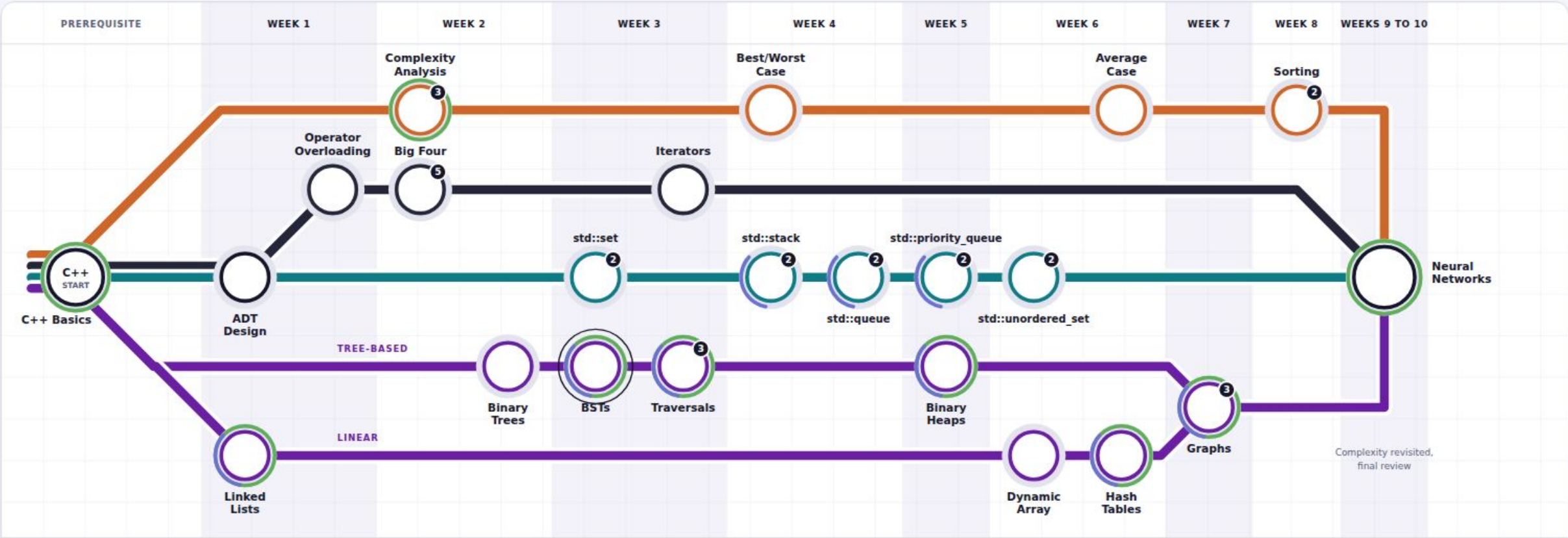
Click a station for its subtopics, reading, and coursework.

C++

ADTs

Theory

Data Structures



Complexity revisited,
final review

WHY THIS COURSE

Why code data structures when AI can do it?

- Coding is an expression of clear understanding, ability to think from first principles.
- Real technical interviews require that demonstrate your problem solving skills and articulate your approach.
- Opportunity to practice these skills in CS24.
- Mock interviews: Intro (2m) → Whiteboard (10m) → Coding (13m) → Feedback (5m)
- Caliber platform: New platform designed and built by UCSB students for formative practice.

70% OF YOUR GRADE: PAPER EXAMS + LIVE INTERVIEWS, NO AI

BEFORE WE START

Logistics Overview

- Grading: · Practice (Leetcode + Caliber) 10% · Labs + PAs: 20%
 - Quizzes 20% · Mock Interview 15% · Final 35%
- 70% of your grade is paper exams and live coding interviews: no AI there
- Textbook: Savitch. Pre-reading is listed on the course site before each lecture
- Mock interviews run in person. Schedule via Caliber.
- Read the syllabus on the course website: <https://ucsb-cs24.github.io/f26/>

About lectures

- Lectures are problem-solving sessions. Ask questions, discuss with neighbors, answer via iClicker.
- Why interactive? You learn by doing—e.g., tracing pointers today.
- Take a moment to introduce yourself to the people sitting near you.
 - Talk about...
 - your background,
 - experience in CS so far, and
 - what you hope to get out of this class!

Quick gauge before we start

How comfortable are you with pointers and dynamic memory in C++ right now?

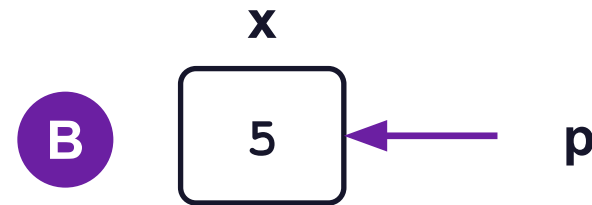
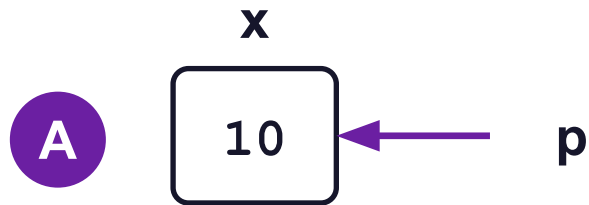
- A — Never used them
- B — Seen them, haven't written much
- C — Comfortable with the basics
- D — Very comfortable



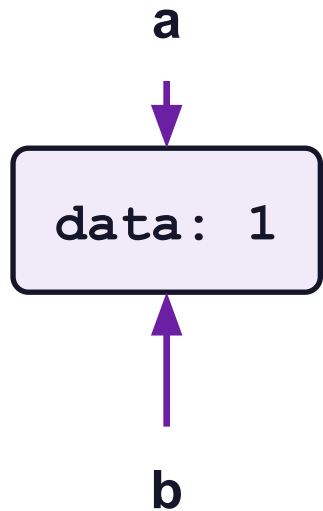
<https://join.iclicker.com/BCRL>

Question 1

```
int x = 5;
int* p = &x;
*p = 10;
// which diagram matches memory now?
```



Question 2

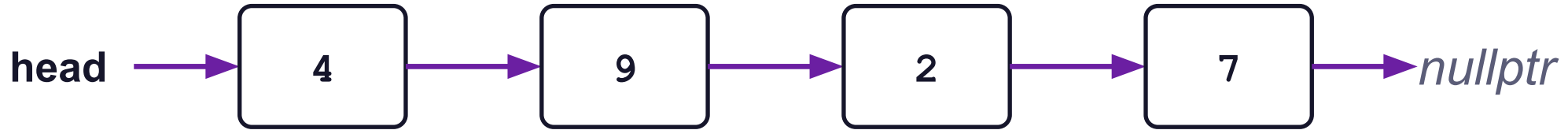


```
Node* a = new Node(1);  
Node* b = a;  
b->data = 99;  
  
// what is a->data now?
```

- A — 1
- B — 99
- C — a crash
- D — undefined

NEW STRUCTURE

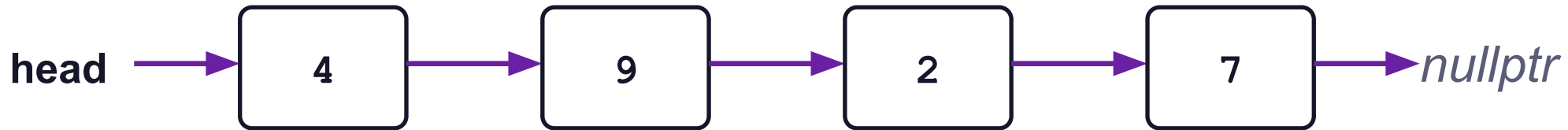
Anatomy of a linked list



```
struct Node {  
    int data;  
    Node* next;  
};
```

- head is the only pointer we store
- Everything else: follow →next until nullptr

Which code adds a node to the head?



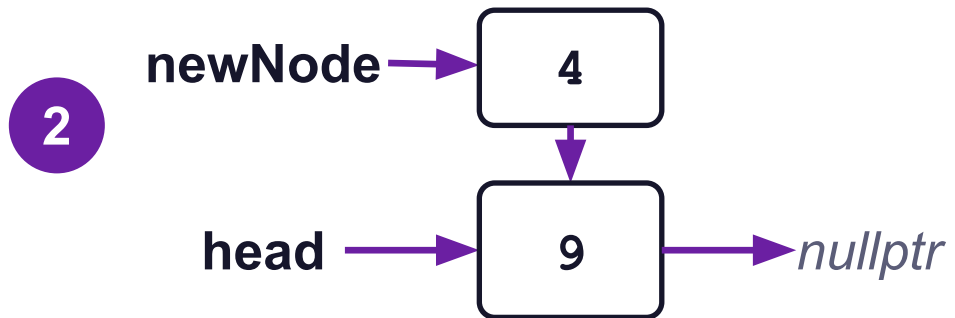
```
struct Node {  
    int data;  
    Node* next;  
};
```

```
Node* newNode = new Node(value);
```

- A) `newNode->next = head; head = newNode;`
- B) `head = newNode; newNode->next = head;`
- C) `newNode->next = newNode; head = newNode;`
- D) `head->next = newNode;`

WALKTHROUGH

Adding a node to the head



THE PROBLEM

Reverse a linked list

```
Given the head of a singly linked list,  
reverse the list, and return the new head.
```

Example:

```
Input:    1 -> 2 -> 3 -> 4 -> 5 -> nullptr
```

```
Output:   5 -> 4 -> 3 -> 2 -> 1 -> nullptr
```

A classic interview question, the same shape you'll see on LeetCode.

Reverse a Linked List

Mapped to the mock-interview rubric

- ☐ Restate + edge case
- ☐ Test case: in $\rightarrow$ out
- ☐ Recursive or iterative?
- ☐ Derive it, think aloud
- ☐ Name the base case
- ☐ Name the subproblem(s)
- ☐ Combine into the answer
- ☒ Complexity + adapt

(whiteboard your solution)

CONCEPT

What is an Abstract Data Type (ADT)?

- Need a collection that grows and shrinks?
- Use C++ ADTs : **std::vector** or **std::list**
- An ADT is defined by what it does (the interface), hiding the implementation details
- Lab01: turn the chain of nodes into a first-class ADT like **std::list**. That's a custom linked list class that you will implement.

CustomList, our ADT:

- `push_back(value)`
- `print()`

(hides: head, tail, Node internals)

SETUP FOR NEXT TIME

Meet Complex

```
class Complex {  
public:  
    Complex(double r, double i);  
    double getReal() const;  
    double getImag() const;  
private:  
    double real, imag;  
};
```

The complex class is an ADT for representing numbers of the form $a + jb$

a is the real part

b is the imaginary part

Is the output of this code deterministic?

```
int main(){  
    Complex p;  
  
    p.conjugate();  
    p.print();  
}
```

```
class Complex  
{  
private:  
    double real;  
    double imag;  
public:  
    double getMagnitude() const;  
    double getReal() const;  
    double getImaginary() const;  
    void print() const;  
    void conjugate();  
    void setReal(double r);  
    void setImag(double r);  
};
```

- A. Yes
- B. No
- C. I am not sure . . .

Is the output of this code deterministic?

```
int main(){  
    Complex p;  
  
    p.conjugate();  
    p.print();  
}
```

```
class Complex  
{  
private:  
    double real;  
    double imag;  
public:  
    double getMagnitude() const;  
    double getReal() const;  
    double getImaginary() const;  
    void print() const;  
    void conjugate();  
    void setReal(double r);  
    void setImag(double r);  
};
```

- A. Yes
- B. No, because the default constructor is an empty routine. p gets random (junk values)**
- C. I am not sure . . .

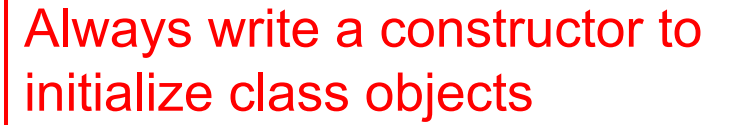
Is the output of this code deterministic?

```
int main(){  
    Complex p;  
  
    p.conjugate();  
    p.print();  
}
```

- A. Yes
- B. No, because the default constructor is an empty routine. p gets random (junk values)**
- C. I am not sure . . .

```
class Complex  
{  
private:  
    double real;  
    double imag;  
public:  
    Complex(): real(0), imag(0){}  
    double getMagnitude() const;  
    double getReal() const;  
    double getImaginary() const;  
    void print() const;  
    void conjugate();  
    void setReal(double r);  
    void setImag(double r);  
};
```

Always write a constructor to initialize class objects



The constructor can take parameters

```
int main(){  
    Complex p(1, 2);  
  
    p.conjugate();  
    p.print();  
}
```

```
class Complex  
{  
private:  
    double real;  
    double imag;  
public:  
    Complex(double re, double im): real(re), imag(im){  
        double getMagnitude() const;  
        double getReal() const;  
        double getImaginary() const;  
        void print() const;  
        void conjugate();  
        void setReal(double r);  
        void setImag(double r);  
};
```

Parameterized constructor



Will this code compile?

```
int main(){  
    Complex p;  
    Complex w(1, 2);  
    p = w;  
    p.conjugate();  
    p.print();  
}
```

- A. Yes
- B. No
- C. I am not sure . . .

```
class Complex  
{  
private:  
    double real;  
    double imag;  
public:  
    Complex(double re, double im): real(re), imag(im){}  
    double getMagnitude() const;  
    double getReal() const;  
    double getImaginary() const;  
    void print() const;  
    void conjugate();  
    void setReal(double r);  
    void setImag(double r);  
};
```

Will this code compile?

```
int main(){  
    Complex p;  
    Complex w(1, 2);  
    p = w;  
    p.conjugate();  
    p.print();  
}
```

```
class Complex  
{  
private:  
    double real;  
    double imag;  
public:  
    Complex(double re, double im):real(re), imag(im){}  
    double getMagnitude() const;  
    double getReal() const;  
    double getImaginary() const;  
    void print() const;  
    void conjugate();  
    void setReal(double r);  
    void setImag(double r);  
};
```

- A. Yes
- B. No: We need the parametrized constructor to take default values
- C. I am not sure . . .

What is the output?

```
int main(){
    Complex p;
    Complex w(1, 2);
    p = w;
    p.conjugate();
    p.print();
    Complex z(p);
    z.print();
}
```

```
class Complex
{
private:
    double real;
    double imag;
public:
    Complex(double re = 0, double im = 0):real(re), imag(im){}
    double getMagnitude() const;
    double getReal() const;
    double getImaginary() const;
    void print() const;
    void conjugate();
    void setReal(double r);
    void setImag(double r);
};
```

Parameterized Constructor with default values provides additional flexibility!



Every line of code is a function call!!! Some are implicit.

```
int main(){
    Complex p;
    Complex w(1, 2);
    p = w;
    p.conjugate();
    p.print();
    Complex z = p;
    z.print();
}
```

```
class Complex
{
private:
    double real;
    double imag;
public:
    Complex(double re = 0, double im = 0):real(re), imag(im){}
    double getMagnitude() const;
    double getReal() const;
    double getImaginary() const;
    void print() const;
    void conjugate();
    void setReal(double r);
    void setImag(double r);
};
```

Critique customList ADT in main.cpp

1 Find at least 3 problems with the flawed CustomList in main.cpp.

- Open main.cpp: this is the version we will NOT keep
- Work with the person next to you, 9 minutes
- Look for: memory issues, slow operations, missing safety checks
- Write the constructor and push_back, live

Constructor + push_back

```
CustomList::CustomList() {  
    head = nullptr;  
    tail = nullptr;  
}  
  
void CustomList::push_back(int val) {  
    // why was push_back slow?  
    // ... build it with tail  
}
```

- why is main.cpp's push_back slow?

ACTIVITY 2 · ON YOUR OWN

Implement print()

2

Write CustomList::print() yourself.

```
void CustomList::print() const {  
    // your code here  
}
```

Copy Constructor Memory diagram

```
CustomList list2 = list1;  
  
// ... what just happened?
```

Homework: Draw exactly what this looks like in memory

BEFORE NEXT TIME

Read, answer, draw, and code

- Read: Savitch 4.3–4.4 (operator overloading) and 5.1, 5.3 (constructors/destructors, Rule of Three)
- Write an answer: If `x` is of type `Complex`, which function does `cout << x` actually call, and why?
- Write an answer: does `Complex` need a destructor? Why or why not?
- Draw and predict: the diagram for `CustomList list2 = list1`? What problems could arise as a result of the above operation?
- Code: Rewrite `reverse()` for a linked list iteratively (`prev/curr`, a while loop)